Linux User Theory

DEVOPS-002 · Task 1 - Understand Linux Users & Groups · Sprint 9 (29/6 - 12/7)

Goal

Learn how Linux manages users and permissions.

1. What is a User?

In Linux, a user is an account that identifies who is interacting with the system. Every person or service that uses the machine is represented by a user account, and every file, process, and running program is owned by a specific user. This ownership is how Linux keeps track of who created something and who is allowed to access it.

Internally, each user is identified by a numeric User ID (UID) rather than by their username. Every account also has a home directory (typically **/home/username**) for personal files, and a default shell (such as **/bin/bash**) that runs when the user logs in. Running the command **id** shows a user's UID, primary group ID, and any secondary groups they belong to.

2. What is Root User?

The root user is the superuser account on a Linux system, always assigned **UID 0**. Unlike normal users, root is not restricted by the standard permission system - it can read, write, or delete any file, install or remove software, create and modify other user accounts, and change core system settings.

Because of this unrestricted power, logging in directly as root is discouraged in practice. Administrators typically use a normal account and prefix individual commands with **sudo** ("superuser do") to gain root privileges only when needed, which keeps a record of who ran what and limits the damage a mistake can cause.

3. What is a Group?

A group is a collection of user accounts that share a common set of permissions. Rather than granting access to a file or resource one user at a time, an administrator can grant it to a group, and every current and future member of that group automatically receives that access.

Every user belongs to a **primary group** (usually created automatically with their account) and can also belong to one or more **secondary groups**. Group membership is identified by a numeric Group ID (GID) and can be viewed with the **groups** or **id** command.

4. Why are Groups required?

- **Scalability** - permissions can be updated for an entire team by changing one group, instead of editing every user account individually.
- **Controlled shared access** - lets multiple users collaborate on the same files or directories, e.g. a shared project folder for a "developers" group.
- **Fewer mistakes** - reduces the chance of forgetting to update permissions for one user out of many when access needs change.

- **Role-based access** - supports organizing accounts by function (admins, developers, testers), which mirrors how real teams are structured.

5. Difference between Root and Normal User

Aspect	Root User	Normal User
UID	0	1000 or higher (typically)
System access	Unrestricted; bypasses permission checks	Limited to own files and permitted areas
Making changes	Can install, modify or delete anything directly	Needs sudo and explicit permission for admin tasks
Shell prompt	#	$
Risk if misused	Can break or compromise the entire system	Impact generally limited to that user's own files
Typical use	System administration and maintenance	Everyday, day-to-day tasks

6. Why does Linux use permissions?

Linux was designed from the ground up as a multi-user operating system, meaning many people and processes can use the same machine at once. Permissions exist to make that possible safely:

- **Security** - stops one user from reading, modifying or deleting another user's files, or tampering with critical system files.
- **Multi-user support** - allows many accounts to coexist on one machine without interfering with each other.
- **Stability** - prevents accidental or malicious changes that could crash or compromise the system.
- **Accountability** - ties every action to a specific user, which supports auditing and troubleshooting.

Permissions are applied at three levels - **owner, group, and others** - and each level can be granted a combination of the following rights:

Permission	Symbol	Meaning
Read	r	View a file's contents, or list a directory's contents
Write	w	Modify a file, or add/remove files inside a directory
Execute	x	Run a file as a program, or enter (cd into) a directory